SWE 539 · Software Metrics

Complete course summary · ↓ lower is better · ↑ higher is better

KFUPM · Dr. Mohammad Alshayeb
Ahmed Abualrob
ahmadrob.github.io/SWE-539-Exam-Kit

1 · C&K SUITE Chidamber & Kemerer 1994

WMC ↓ = Σ c_i over local methods. All c_i = 1 ⇒ WMC = n (# methods).
More time/effort · more impact on children (inherit all) · app-specific ⇒ less reuse.

DIT = depth from root, **root = 0**. Multiple inheritance ⇒ **max** path to root.
Deeper: inherits more ⇒ harder to predict · more design complexity · + more reuse.

NOC = # **immediate** subclasses.
More: + reuse · improper abstraction / subclass misuse · more influence ⇒ more testing.

CBO ↓ = |classes it uses ∪ classes that use it|. Both directions, **set** (count once). Via calls, field access, inheritance, args, return types. Unit: class.
High: hurts modularity & reuse · change-sensitive · rigorous testing.

RFC ↓ = |M ∪ R|. M = own methods, R = distinct methods they call. Call to own method adds 0.
High: hard to test/debug · complex · worst case plans test time.

LCOM ↓ = |P| − |Q| if |P| > |Q|, else 0. P = pairs sharing **no** attribute, Q = pairs sharing ≥ 1.
Cohesion promotes encapsulation · lacking ⇒ split class · more errors.

Worked examples

- **CBO Ex.1** (A◇–B, A⇢C, B–C, C–D): A 2 · B 2 · C 3 · D 1.
- **CBO(C) = 3**: C extends SuperC (uses its field and method), calls B 3×, and A calls C ⇒ {SuperC, B, A}; B counts once.
- **Bookshop** (Customer → Orders, CreditCardInfo; Order → Books; RegularBook, EBook extend Book): Customer 2 · Order 2 · CreditCardInfo 1 · Book 3 · RegularBook 1 · EBook 1.
- **RFC:** A{A1,A2,A3} → B1, B2, C1 ⇒ **6**. A.m1..m4 → B.m1, B.m2, C.m1, D.m1, D.m2 ⇒ **9**. A.m1→B.m2, A.m2→C.m1, A.m3→A.m4 (own, adds 0) ⇒ **6**.
- **LCOM:** I1{a,b,c,d,e} I2{a,b,e} I3{x,y,z} ⇒ P 2, Q 1 ⇒ **1**. Weakness: many different classes all get 0.

2 · LI SUITE Li 1998 · fixes C&K

C&K ISSUE	LI METRIC
DIT ambiguous with multiple roots (B extends C and D, D extends E: DIT 1 or 2?) · DIT(A) = DIT(B) = 2 though B has more ancestors	**NAC** = # **all** ancestors (unit: class)
NOC counts only immediate children	**NDC** = # **all** descendants
WMC mixes count and complexity	**NLM** = # local methods (unit: method) · **CMC** = Σ internal complexity (unit: LOC / McCabe paths)
CBO: coupling has 3 forms (inheritance, ADT, message passing)	**CTA = DAC**: # classes used as **attribute types** · **CTM = MPC**: # **different** messages sent out, excluding locally created objects

CTA: `class B {A anA;}` ⇒ CTA(B) = 1. CTM: `A1(B* b){b->B1();}` ⇒ CTM(A) = 1.

Example tree: roots A, J · B<A · D<B · E,F<D · K<J · **C<A and K** · G,H,I<C.

CLASS	DIT	NOC	NAC	NDC
A	0	2	0	8
J	0	1	0	5
B · K	1 · 1	1 · 1	1 · 1	3 · 4
D	2	2	2	2
C	2	3	3	3
E, F	3	0	3	0
G, H, I	3	0	4	0

Other coupling (Al Dallal): CBO, RFC, DAC, MPC · **CBO_U** classes it uses · **CBO_IUB** classes that use it (both excl. inherited) · **OCMEC** distinct parameter types.

TRAP Single inheritance: DIT = NAC. Root: DIT = NAC = 0. Leaf: NOC = NDC = 0. DIT(C) = 2 (longest path) but NAC(C) = 3.

3 · COHESION FAMILY

Example class: m1{a1,a2} · m2{a1} · m3{a3} · m4{a4} · calls m2→m3, m4→m3. m = a = 4 · pairs = m(m−1)/2 = 6 · Σp(A_i) = # method–attribute links = 5.

METRIC	RULE	VALUE
LCOM1 ↓	|P|	5
LCOM2 ↓	max(|P| − |Q|, 0)	4
LCOM3 ↓ Li & Henry 93	# components; edge = share an attribute	3
LCOM4 ↓ Hitz & Montazeri 95	# components; edge = share attribute **or call**	1
LCOM5 ↓ Henderson-Sellers 96	$\frac{m - \Sigma p(A_i)/a}{m - 1}$	11/12
Coh ↑ Briand 98	$\frac{\Sigma p(A_i)}{m \cdot a}$	5/16
TCC ↑ Bieman & Kang 95	direct pairs / pairs. Direct = share an attribute, counting attributes of methods it invokes (directly or transitively)	4/6
LCC ↑ Bieman & Kang 95	(direct + indirect) / pairs. Indirect = linked through a chain (same component)	6/6
CC ↑ Bonja & Kidanmariam 06	ΣSim / pairs, Sim = $\lvert I_i \cap I_j \rvert$ / $\lvert I_i \cup I_j \rvert$. Only Sim(m1,m2) = 1/2	1/12

TCC vs LCC example (no calls): m1{a1,a2} m2{a1,a3} m3{a3} m4{a4} ⇒ TCC **2/6** (m1–m2, m2–m3), LCC **3/6** (+ m1–m3 via m2).

TRAP Calls count only in **LCOM4, TCC, LCC**. LCC ≥ TCC. LCOM5: divide Σ by a first. Pairs m(m−1)/2 here, but package cohesion uses n(n−1).

COHESION TYPE (BEST → WORST)	KEPT TOGETHER
1 Functional	one computation, no side effects
2 Layer	related services, strict hierarchy (higher → lower)
3 Communicational	operate on the same data (good classes)
4 Sequential	output of one = input of next
5 Procedural	called one after another
6 Temporal	same phase (initialization, termination)
7 Utility	related utilities, no stronger grouping

COUPLING TYPE (WORST → BEST)	FIX
1 Content: modify another's internals	always avoid
2 Common: global variables	severely restrict
3 Control: flag (if) controls another	polymorphism
4 Stamp: app class as argument	interface, superclass, few data items
5 Data: simple-data arguments	fewer arguments
6 Routine call	encapsulate repeated sequences
7 Type use: global data type	superclass or interface
8 Inclusion / import	remove if not needed
9 External: OS, libraries, hardware	fewer dependent places

Taxonomy (Fregnan 2019): structural (procedural; OO = interaction, component [hidden, scattered, specified], inheritance [modification, refinement, extension]) · dynamic (import, export) · semantic · logical.

4 · PACKAGE METRICS R. Martin

Package cohesion ↑ = actual class couplings / n(n − 1), n = classes in package. Range 0–1. Example: B→A, C→A, C↔D (counts 2), C→E, D◇E = 6 directed couplings, n = 5 ⇒ 6 / (5·4) = **0.3**.

Ca (afferent, fan-**in**) = # classes **outside** that depend on classes **inside**.
Ce (efferent, fan-**out**) = # classes **inside** that depend on classes **outside**.

I = $\frac{Ce}{Ca + Ce}$ · 0 = completely stable · 1 = completely unstable.
Example Pc{t,u}: q extends t, r and s use u (all outside) ⇒ Ca 3; u extends v (outside) ⇒ Ce 1 ⇒ **1/4**.

Stability = amount of work to make a change, **not** change frequency. Many dependents ⇒ hard to change ⇒ stable (X: 3 in ⇒ I = 0; Y: 3 out ⇒ I = 1).

TRAP Count **classes**, not arrows. Ce on top. Read class-level arrows, not dashed package arrows.

5 · REFACTORING Fowler

Code smell: not a bug; a surface indication of a deeper problem; signals **technical debt** = cost of extra work from a quick solution now. Debt causes: business pressure · not knowing consequences · delayed refactoring.

Refactoring: change internal structure, keep external behaviour (Fowler); improve non-functional quality (Beck); disciplined, minimises bugs. Agile and any process.

3 hats, one at a time: coding (features) · testing (add tests) · refactoring (no new features or tests).

Why: better design (easier change) · understandable · find bugs · clean code (obvious, no duplication, passes tests, minimal parts). Design right first time is very hard.

When: adding a feature · fixing a bug · code review (pair with author). **Not**: close deadlines · prematurely published interfaces · unstable rubbish code. Cunningham: unfinished refactoring = debt.

Activities (Mens & Tourwé): identify (smells) → choose → guarantee behaviour → apply → assess quality → keep artifacts consistent. Small steps, test after each.

Own refactoring: change → no external change → small steps (compile/test points) → do + note → redo + refine → 2–3 times.

Typical: class (add subclass, rename, remove) · method (add, rename, remove, push down/up, add parameter, move to component, extract) · attribute (add, rename, remove, push down, pull up, create accessors, abstract).

CLONE	COPIES DIFFER IN
I	layout, white space, comments
II	+ identifiers, literals, types
III	+ statements changed / added / removed
IV	same computation, different syntax (for vs while)

Tools: CCFinder, CodePro Analytix, NiCad, Deckard, CloneDr, Cpdetector, Simian, SimScan, JPlag, iClones.

Duplicates in: same class → Extract Method · sibling subclasses → Extract + Pull Up Method · unrelated (instance) → common superclass · unrelated (static) → utility class.

REFACTORING	WHEN → WHAT
Extract Method	groupable fragment → method named by intention. Steps: create · copy · locals → param / return / declare · compile · replace with call · compile + test
Inline Method	body as clear as name → put body in callers, delete
Replace Temp w/ Query	temp holds expression → method. Steps: temp assigned once · extract RHS · replace refs · remove temp · compile + test
Move Method	uses another class more → move; old one delegates or is removed
Extract Class	one class does two jobs → new class (Person → TelephoneNumber)

SMELL WITHIN CLASS	REFACTORINGS
Comments (why, not what)	Extract / Rename Method, Introduce Assertion
Long Method (>20 bad, <10 good)	Extract Method, Replace Temp w/ Query, Intro Param Object, Preserve Whole Object, Replace Method w/ Method Object
Long Parameter List (object missing)	Replace Param w/ Method, Preserve Whole Object, Intro Param Object
Duplicated Code (worst)	Extract Method, Pull Up Field, Form Template Method, Substitute Algorithm
Large Class (too many responsibilities)	Extract Class / Subclass, Move Method
Type Embedded in Name · Uncommunicative · Inconsistent Names	Rename Method (`schedule.add(course)`)
Dead Code	delete

SMELL BETWEEN CLASSES	REFACTORINGS
Primitive Obsession (money, date range)	Replace Data Value w/ Object; Type Code → Class / Subclasses / State-Strategy; Extract Class; Intro Param Object; Replace Array w/ Object
Data Class (only fields, DTO)	Move Method (behaviour in)
Data Clumps	Extract Class → Intro Param Object / Preserve Whole Object
Refused Bequest (LSP)	usually fine; sibling + Push Down Method/Field; worst → Replace Inheritance w/ Delegation
Inappropriate Intimacy	Move Method/Field, Bi → Unidirectional, Extract Class, Hide Delegate, Replace Inheritance w/ Delegation
Lazy Class (does too little)	Collapse Hierarchy, Inline Class
Feature Envy	(Extract +) Move Method
Message Chains	Hide Delegate; Extract + Move Method
Middle Man	Remove Middle Man; Inline Method; Replace Delegation w/ Inheritance
Divergent Change: 1 class, many reasons	Extract Class
Shotgun Surgery: 1 change, many classes	Move Method/Field, Inline Class
Parallel Inheritance (special shotgun)	Move Method/Field into one hierarchy

Obstacles: complexity · error risk (tests) · culture ("pay for features") · performance · time pressure (after delivery).

Model refactoring (UML, MDD): + visualization (smell detection) · efficiency · language independence · less complexity. Class diagrams (>60% of Fowler): addition, removal, move, generalization, specialization. State machines: group states, fold/unfold transitions, move into/out of composite. To patterns: Extract Composite.

Tools: IntelliJ, Eclipse, NetBeans, JRefactory, jFactor, RefactorIT, Xrefactory. Research: detection, correction, prioritization.

6 · CH1 · MEASUREMENT

Measurement = assign numbers/symbols to **attributes** of **entities** by clear rules. Entity = object or event; attribute = property. Accuracy = instrument + definition.

Direct: distance, # classes. **Indirect** (combined): speed, LOC/h, bugs/KLOC, league points.

Galileo: make measurable. DeMarco: can't control what you can't measure. **Gilb fuzzy targets**: no clear goals ⇒ goals not met clearly.

Why: understand (baselines) → control (predict, adjust) → improve. Every measure needs a clear goal. Managers: cost, productivity, quality, satisfaction, improvement. Engineers: testable requirements, faults found, goals met, future.

Who measures: developers (requirements, design quality, code ready to test) · managers (delivery date, budget) · customers (requirements met, quality) · maintainers (what to upgrade).

Scope: quality models (operation: usability, reliability, efficiency; revision: reusability, maintainability, portability, testability) · reliability · security · structural/complexity · capability maturity · management by metrics · method/tool evaluation.

Metrics show: whether process/product improves · baselines and goals · causes of defects · requirements likely to change · error-prone modules · testing needed per module.

7 · CH2 · BASICS & SCALES

Representational theory: data represent the attribute; manipulation preserves observed relations. Empirical relations: unary, binary, ternary. Measurement = mapping; measure = assigned number. Unagreed relations (preference: tastier meal) ⇏ rating formats.

> **Representation condition**: empirical relation ⇔ numerical relation (both ways). Tall ⇔ M > 70 · much taller ⇔ M(A) > M(B) + 15 · shoulders: 0.7M(A) + 0.8M(C) > M(B). Facts: F taller than W; F, W tall, P not; F, W much taller than P; F not much taller than W; P on W's shoulders higher than F. Valid W72 F84 P42; **P60 breaks** (72 > 75 false).

Stages: attribute → empirical relations → numerical relations → mapping → check representation condition. Failure criticality: add order ⇒ M(data loss) > M(incorrect) > M(delayed). **Richer system ⇒ fewer valid measures.**

Models: LOC needs rules (subroutines, multi-statement lines, comments, declarations, languages, platforms). **McCabe V(G) = e − n + 2 = d + 1** (4 decisions ⇒ 5); can mislead.

Direct: LOC, test hours, # defects, months. **Indirect**: LOC/PM, defect density, defect detection density, requirements stability, test effectiveness, system spoilage. Faults per area (direct) vs faults/KLOC (indirect): a 4 KLOC area of a 2-million-line system is the real problem.

Admissible transformation = map between acceptable measures. More restrictive ⇒ more sophisticated scale (richness: nominal < ordinal < interval < ratio < absolute).

SCALE	ADMISSIBLE M′	OPS	EXAMPLE
Nominal	any 1-1	mode, count	fault type, language
Ordinal	monotonic ↑	+ median	severity, pain faces
Interval	aM + b, a>0	+ mean, ±	°C, dates
Ratio	aM, a>0	+ ×÷, zero	LOC, ms, TB, SR
Absolute	M only	all	# failures, # people

Ordinal 1,2,3,4,10 ✓ · 1,2,3,4,**4** ✗ · 1,3,2,4,10 ✗. Interval 2,4,6,8 ✓ · 1,3,5,8 ✗. Ratio 3,6,9,12 ✓ · 1,2,4,8 ✗.

> **Meaningful** = truth value invariant under admissible transformations (≠ true). ✔ ≥ 100 errors · twice as long to fix · twice as tall · temp. difference doubled · "week has 8 days" (false). ✖ cost ≥ 100 (no unit) · twice as hot (40 vs 20 °C ≠ 104 vs 68 °F) · twice as critical (ordinal).

Ordinal mean flips: scale 1–5; X = trivial, simple ×2, moderate, incomprehensible; Y = simple, moderate ×3, complex ×3. X mean 2.6 → 3.6 when incomprehensible = 10; Y = 23/7 ≈ 3.29. Medians X 2, Y 3 stay ⇒ use **median**.

Central tendency: mean, median, mode · **dispersion**: max, min, std. dev. Attributes not directly measurable ⇒ vectors of measures combined into an indirect measure.

Objective: same value for anyone. **Subjective**: depends on person/environment (diving vs running); still useful.

8 · CH3 · GQM FRAMEWORK

ENTITY	INTERNAL	EXTERNAL
Process (activities)	duration, effort, # incidents (only direct ones)	cost-effectiveness, inspection effectiveness
Product (any artifact)	LOC, complexity, coupling	reliability, usability, failures, search time
Resource (personnel, material, methods)	team size, price	productivity, tool usability

Internal = entity alone, no execution. **External** = with environment, harder, late ⇒ predict from internal. Resources: magnitude, cost, quality. Research: link internal ↔ external; measure directly.

Samples: development (elapsed days, milestone dates, staff-hours) · test (tests scheduled, executed, passed) · system size (modules, DFD bubbles, function points, SLOC, memory) · defect density (per KLOC, per FP) · staff (team size, years of experience) · time (start/due dates).

AT&T inspection: effort/KLOC, faults/KLOC · testing: errors, duration, cost per sub-process · AI review: defects/1000 lines, comment precision, cost/defect.

> **GQM**: goals → questions → metrics. Template: Analyze ⟨object⟩ for the purpose of ⟨purpose⟩ with respect to ⟨quality focus⟩ from the view of ⟨perspective⟩ in the context of ⟨context⟩. Alt: Purpose · Perspective · Environment. **Metric answering no question ⇒ drop.**

Unit-test example: M1 tests/week · M2 time per test · M3 tests that find a defect · M4 unit-test defects · M5 integration defects unit test missed · M6 tests added. Q1 test time → M1, M2 · Q2 effectiveness → M1, M2, M4 · Q3 last added tests → M1, M3, M5, M6 (one metric, many questions). Other trees: coding standard, inspection, reliability, maintainability, LLM-generated tests.

CMMI	LEVEL
1 Initial	ad hoc, heroics
2 Managed	planned, monitored, reviewed
3 Defined	standards, procedures, tools
4 Quant. managed	quantitative quality objectives
5 Optimizing	continual quantitative improvement

Level 2: 7 process areas, 15 goals: configuration mgmt · measurement & analysis · monitoring & control · planning · process & quality assurance · requirements mgmt · supplier agreements. CM: identify items, CM system, baselines, track/control changes, records, audits.

High-level goals: productivity, quality, risk. Productivity subgoals: staff skills, managerial skills, technology; early problem detection, right technology, reuse.

Maturity → metrics ("requirements maintainable?"): L1 count reqs + changes · L2 + type, changes per type · L3 + traceability. More mature ⇒ richer measures.

9 · EMPIRICAL INVESTIGATION

Studies **support or refute**, never prove. Theory: cause → effect construct; observation: treatment (independent) → output (dependent).

Design principles: level of control (decides type) · goals & hypotheses · variable control · validity threats · human subjects.

Exploratory (qualitative: interviews, observation) vs **explanatory** (quantitative: cause–effect, statistics).

TYPE	SCALE	KEY
Controlled experiment	small	high control, planned, academia
Case study (= quasi-exp.)	typical	document a typical project; planned or retrospective
Survey	large	retrospective, no manipulation

Quasi-experiment: no random assignment (IV is a participant trait: experience, gender).

Choose: retrospective ⇒ survey / case study · high control ⇒ experiment (risky, e.g. safety-critical ⇒ case study) · no replication ⇒ case study · cheap replication ⇒ experiment. Experiment = high control & replication, low difficulty & cost; more generalizable.

Hypothesis: quantifiable (fewer defects/KLOC). Surrogate measures risk construct validity. **H0** = no difference; reject only if $p < 5\%$; then HA1 / HA2.

State variable (independent). Experiment samples **over** it; case study samples **from** it (typical value). Control language C vs experimental Java, all else equal. NASA: Ada beat FORTRAN on 3rd use. No control of subject selection ⇒ case study. Toy or field setting.

VALIDITY	THREAT EXAMPLE
Conclusion (statistics)	wrong test, small sample, too many variables
Construct (right measure)	faults/KLOC timing
Internal (real cause)	hidden variables, no causal theory, B causes A, repeated testing
External (generalize)	students vs professionals

Design to minimise threats **before** the study, not after.

Human subjects (45 CFR 46.102(d)): data by interaction, or identifiable private info. Main risk: privacy. **IRB**: protections (encrypt, destroy IDs) and likely impact.